SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems COURSE**
CODE: **CSA05**

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I ADHAVAN CHANDRASEKAR(192521209) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ADHAVAN CHANDRASEKAR(192521209)

ANSWERS:

1) **For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.**

CODE:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE STUDENT_COURSE (
    ->     SID INT,
    ->     SName VARCHAR(30),
    ->     CID VARCHAR(10),
    ->     CName VARCHAR(30),
    ->     Instructor VARCHAR(30),
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY (SID, CID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO STUDENT_COURSE VALUES
    -> (101, 'Arun',  'C101', 'DBMS',     'Dr.Ravi',   'A'),
    -> (101, 'Arun',  'C102', 'DSA',      'Dr.Meena',  'B'),
    -> (102, 'Meena', 'C101', 'DBMS',     'Dr.Ravi',   'A'),
    -> (103, 'Kiran', 'C103', 'Networks', 'Dr.Suresh', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE;
+-----+-------+------+----------+------------+-------+
| SID | SName | CID  | CName    | Instructor | Grade |
+-----+-------+------+----------+------------+-------+
| 101 | Arun  | C101 | DBMS     | Dr.Ravi    | A     |
| 101 | Arun  | C102 | DSA      | Dr.Meena   | B     |
| 102 | Meena | C101 | DBMS     | Dr.Ravi    | A     |
| 103 | Kiran | C103 | Networks | Dr.Suresh  | B     |
+-----+-------+------+----------+------------+-------+
4 rows in set (0.00 sec)

mysql>
```

QUERY 1 – Verify SID → SName

```
mysql> SELECT SID, COUNT(DISTINCT SName) AS Name_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY SID;
+-----+------------+
| SID | Name_Count |
+-----+------------+
| 101 |          1 |
| 102 |          1 |
| 103 |          1 |
+-----+------------+
3 rows in set (0.00 sec)

mysql>
```

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

```
mysql> SELECT CID,
    ->        COUNT(DISTINCT CName) AS Course_Count,
    ->        COUNT(DISTINCT Instructor) AS Instructor_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY CID;
+------+--------------+------------------+
| CID  | Course_Count | Instructor_Count |
+------+--------------+------------------+
| C101 |            1 |                1 |
| C102 |            1 |                1 |
| C103 |            1 |                1 |
+------+--------------+------------------+
3 rows in set (0.00 sec)

mysql>
```

Therefore:

- CID → CName
- CID → Instructor

QUERY 3 – Verify Candidate Key (SID, CID)

```
mysql> SELECT SID, CID, COUNT(*) AS Record_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY SID, CID;
+-----+------+--------------+
| SID | CID  | Record_Count |
+-----+------+--------------+
| 101 | C101 |            1 |
| 101 | C102 |            1 |
| 102 | C101 |            1 |
| 103 | C103 |            1 |
+-----+------+--------------+
4 rows in set (0.00 sec)

mysql>
```

Every (SID, CID) combination uniquely identifies one record.
Therefore:

- (SID, CID) → Grade
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- SID → SName
- CID → CName
- CID → Instructor
- (SID, CID) → Grade

CANDIDATE KEY:

- (SID, CID)

2) **Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.**

```
mysql> CREATE TABLE RESEARCHER_PROJECT_UNF (
    ->     ResearcherID VARCHAR(10),
    ->     ResearcherName VARCHAR(30),
    ->     Projects VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCHER_PROJECT_UNF VALUES
    -> ('R101', 'Arun', 'AI, Robotics'),
    -> ('R102', 'Meena', 'IoT'),
    -> ('R103', 'Kiran', 'Cyber Security, Cloud');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCHER_PROJECT_UNF;
+--------------+----------------+----------------------+
| ResearcherID | ResearcherName | Projects             |
+--------------+----------------+----------------------+
| R101         | Arun           | AI, Robotics         |
| R102         | Meena          | IoT                  |
| R103         | Kiran          | Cyber Security, Cloud |
+--------------+----------------+----------------------+
3 rows in set (0.00 sec)

mysql>
```

CONVERT TO 1NF:

Each project is placed in a **separate row**.

```
mysql> CREATE TABLE RESEARCHER_PROJECT_1NF (
    ->     ResearcherID VARCHAR(10),
    ->     ResearcherName VARCHAR(30),
    ->     Project VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCHER_PROJECT_1NF VALUES
    -> ('R101', 'Arun', 'AI'),
    -> ('R101', 'Arun', 'Robotics'),
    -> ('R102', 'Meena', 'IoT'),
    -> ('R103', 'Kiran', 'Cyber Security'),
    -> ('R103', 'Kiran', 'Cloud');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCHER_PROJECT_1NF;
+--------------+----------------+----------------+
| ResearcherID | ResearcherName | Project        |
+--------------+----------------+----------------+
| R101         | Arun           | AI             |
| R101         | Arun           | Robotics       |
| R102         | Meena          | IoT            |
| R103         | Kiran          | Cyber Security |
| R103         | Kiran          | Cloud          |
+--------------+----------------+----------------+
5 rows in set (0.00 sec)

mysql>
```

3) **Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.**

CODE:

```
mysql> CREATE TABLE RESEARCH_RECORD (
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_RECORD VALUES
    -> (101, 'B1', 'C1', 'D1', 'E1'),
    -> (102, 'B2', 'C2', 'D2', 'E2'),
    -> (103, 'B3', 'C3', 'D3', 'E3'),
    -> (104, 'B4', 'C4', 'D4', 'E4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM RESEARCH_RECORD;
+-----+------+------+------+------+
| A   | B    | C    | D    | E    |
+-----+------+------+------+------+
| 101 | B1   | C1   | D1   | E1   |
| 102 | B2   | C2   | D2   | E2   |
| 103 | B3   | C3   | D3   | E3   |
| 104 | B4   | C4   | D4   | E4   |
+-----+------+------+------+------+
4 rows in set (0.00 sec)
```

QUERY – Verify Candidate Key A

```
mysql> CREATE TABLE RESEARCH_RECORD (
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_RECORD VALUES
    -> (101, 'B1', 'C1', 'D1', 'E1'),
    -> (102, 'B2', 'C2', 'D2', 'E2'),
    -> (103, 'B3', 'C3', 'D3', 'E3'),
    -> (104, 'B4', 'C4', 'D4', 'E4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT A,
    ->        COUNT(DISTINCT B) AS B_count,
    ->        COUNT(DISTINCT C) AS C_count,
    ->        COUNT(DISTINCT D) AS D_count,
    ->        COUNT(DISTINCT E) AS E_count
    -> FROM RESEARCH_RECORD
    -> GROUP BY A;
+-----+---------+---------+---------+---------+
| A   | B_count | C_count | D_count | E_count |
+-----+---------+---------+---------+---------+
| 101 |       1 |       1 |       1 |       1 |
| 102 |       1 |       1 |       1 |       1 |
| 103 |       1 |       1 |       1 |       1 |
| 104 |       1 |       1 |       1 |       1 |
+-----+---------+---------+---------+---------+
4 rows in set (0.00 sec)

mysql>
```

A+ = {A,B,C,D,E}

Therefore, Candidate Key = A.

4) **Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.**

```
mysql> CREATE TABLE EMPLOYEE (
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(30),
    ->     DeptID VARCHAR(10),
    ->     DeptName VARCHAR(30),
    ->     ManagerID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (101, 'Arun',  'D01', 'Research', 'M201'),
    -> (102, 'Meena', 'D01', 'Research', 'M201'),
    -> (103, 'Kiran', 'D02', 'Robotics', 'M202'),
    -> (104, 'Ravi',  'D03', 'AI Lab',  'M203');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
+-------+---------+--------+----------+-----------+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-------+---------+--------+----------+-----------+
|   101 | Arun    | D01    | Research | M201      |
|   102 | Meena   | D01    | Research | M201      |
|   103 | Kiran   | D02    | Robotics | M202      |
|   104 | Ravi    | D03    | AI Lab   | M203      |
+-------+---------+--------+----------+-----------+
4 rows in set (0.00 sec)
```

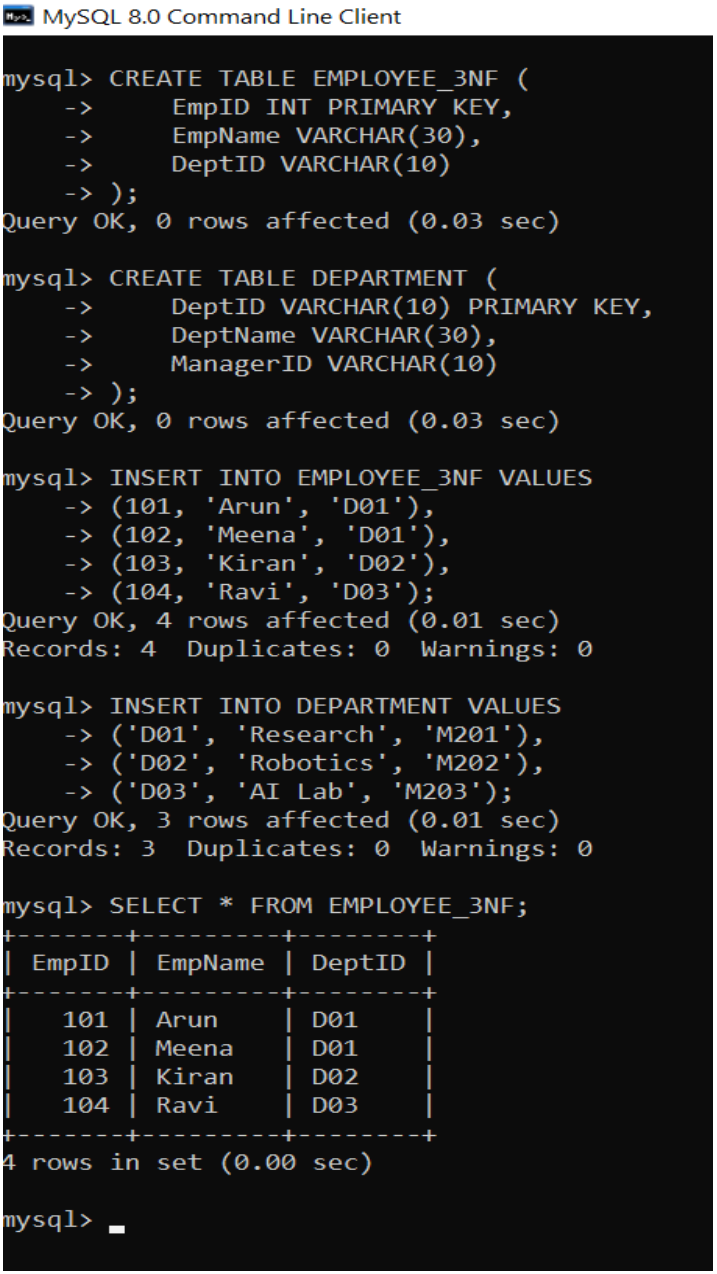
FUNCTIONAL DEPENDENCIES

- EmpID → EmpName, DeptID
- DeptID → DeptName, ManagerID

Here, the transitive dependency is:

- EmpID → DeptID → DeptName, ManagerID

DeptName and ManagerID depend on EmpID **through DeptID**, creating a transitive dependency.



DECOMPOSE INTO 3NF

```
mysql> SELECT * FROM DEPARTMENT;
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
| D01    | Research | M201      |
| D02    | Robotics | M202      |
| D03    | AI Lab   | M203      |
+--------+----------+-----------+
3 rows in set (0.00 sec)
```

5) **Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.**

MySQL 8.0 Command Line Client

```
Database changed
mysql> CREATE TABLE RELATION_ABCD (
    ->     A VARCHAR(10),
    ->     B VARCHAR(10),
    ->     C VARCHAR(10),
    ->     D VARCHAR(10),
    ->     PRIMARY KEY (A, B)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO RELATION_ABCD VALUES
    -> ('A1','B1','C1','D1'),
    -> ('A2','B2','C2','D2'),
    -> ('A3','B3','C3','D3'),
    -> ('A4','B4','C4','D4');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RELATION_ABCD;
+----+----+------+------+
| A  | B  | C    | D    |
+----+----+------+------+
| A1 | B1 | C1   | D1   |
| A2 | B2 | C2   | D2   |
| A3 | B3 | C3   | D3   |
| A4 | B4 | C4   | D4   |
+----+----+------+------+
4 rows in set (0.01 sec)
```

```
mysql> SELECT A, B, COUNT(*) AS Record_Count
    -> FROM RELATION_ABCD
    -> GROUP BY A, B;
+----+----+--------------+
| A  | B  | Record_Count |
+----+----+--------------+
| A1 | B1 |            1 |
| A2 | B2 |            1 |
| A3 | B3 |            1 |
| A4 | B4 |            1 |
+----+----+--------------+
4 rows in set (0.01 sec)

mysql>
```

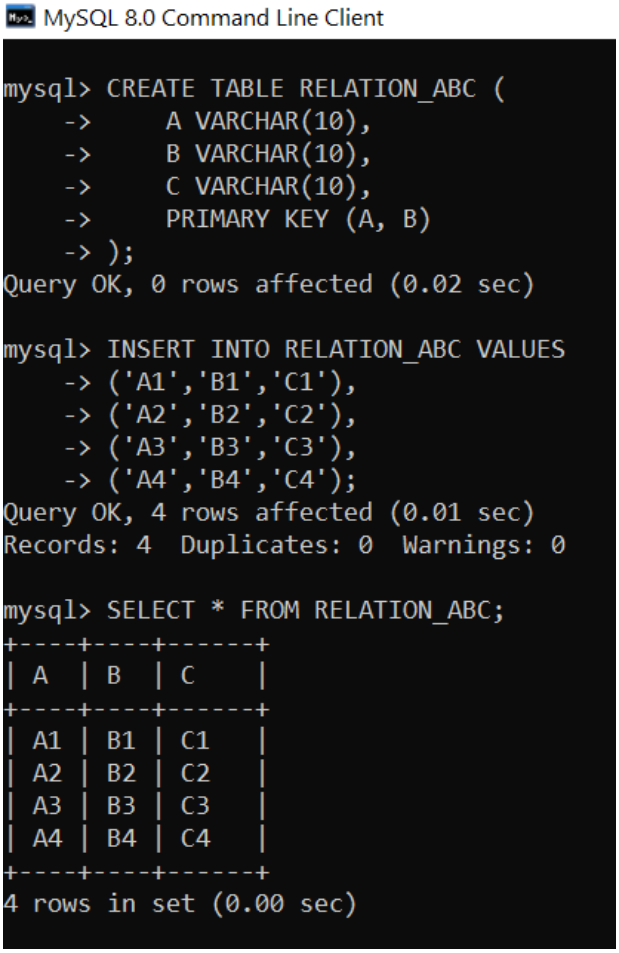
Therefore, AB is a candidate key.

CREATE FIRST BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CD (
    ->     C VARCHAR(10) PRIMARY KEY,
    ->     D VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RELATION_CD VALUES
    -> ('C1','D1'),
    -> ('C2','D2'),
    -> ('C3','D3'),
    -> ('C4','D4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RELATION_CD;
+----+------+
| C  | D    |
+----+------+
| C1 | D1   |
| C2 | D2   |
| C3 | D3   |
| C4 | D4   |
+----+------+
4 rows in set (0.00 sec)
```



From:

- C → D

Decompose into:

- R1(C,D)
- R2(A,B,C)

SECOND BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CA (
    ->     C VARCHAR(10) PRIMARY KEY,
    ->     A VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE RELATION_BC (
    ->     B VARCHAR(10),
    ->     C VARCHAR(10),
    ->     PRIMARY KEY (B, C)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RELATION_CA VALUES
    -> ('C1','A1'),
    -> ('C2','A2'),
    -> ('C3','A3'),
    -> ('C4','A4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO RELATION_BC VALUES
    -> ('B1','C1'),
    -> ('B2','C2'),
    -> ('B3','C3'),
    -> ('B4','C4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM RELATION_CA;
+----+------+
| C  | A    |
+----+------+
| C1 | A1   |
| C2 | A2   |
| C3 | A3   |
| C4 | A4   |
+----+------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM RELATION_BC;
+----+----+
| B  | C  |
+----+----+
| B1 | C1 |
| B2 | C2 |
| B3 | C3 |
| B4 | C4 |
+----+----+
4 rows in set (0.00 sec)
```

Final Decomposition:

- R1(C,D)
- R2(C,A)
- R3(B,C)

6) **Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.**

```
mysql> CREATE TABLE RESEARCH_PROJECT (
    ->     ResearcherID VARCHAR(10),
    ->     ProjectID VARCHAR(10),
    ->     ProjectName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_PROJECT VALUES
    -> ('R101','P101','AI Research'),
    -> ('R102','P102','Robotics'),
    -> ('R103','P103','Cyber Security'),
    -> ('R104','P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCH_PROJECT;
+--------------+-----------+-----------------+
| ResearcherID | ProjectID | ProjectName     |
+--------------+-----------+-----------------+
| R101         | P101      | AI Research     |
| R102         | P102      | Robotics        |
| R103         | P103      | Cyber Security  |
| R104         | P104      | Cloud Computing |
+--------------+-----------+-----------------+
4 rows in set (0.00 sec)
```

CREATE DECOMPOSED RELATIONS

```
mysql> CREATE TABLE RESEARCHER_PROJECT (
    ->     ResearcherID VARCHAR(10),
    ->     ProjectID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE PROJECT_DETAILS (
    ->     ProjectID VARCHAR(10),
    ->     ProjectName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RESEARCHER_PROJECT VALUES
    -> ('R101','P101'),
    -> ('R102','P102'),
    -> ('R103','P103'),
    -> ('R104','P104');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PROJECT_DETAILS VALUES
    -> ('P101','AI Research'),
    -> ('P102','Robotics'),
    -> ('P103','Cyber Security'),
    -> ('P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

CHASE TABLE:

Initial Chase Table:

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	b3
R2	b1	a2	a3

FD: ProjectID → ProjectName

Since both rows have the same ProjectID (a2),
ProjectName takes the common distinguished value (a3).

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	a3
R2	b1	a2	a3

A row contains all distinguished symbols.
Therefore, the decomposition is LOSSLESS.

VERIFY USING SQL JOIN:

```
mysql> SELECT r.ResearcherID,
    ->        r.ProjectID,
    ->        p.ProjectName
    -> FROM RESEARCHER_PROJECT r
    -> JOIN PROJECT_DETAILS p
    -> ON r.ProjectID = p.ProjectID;
+--------------+-----------+-----------------+
| ResearcherID | ProjectID | ProjectName     |
+--------------+-----------+-----------------+
| R101         | P101      | AI Research     |
| R102         | P102      | Robotics        |
| R103         | P103      | Cyber Security  |
| R104         | P104      | Cloud Computing |
+--------------+-----------+-----------------+
4 rows in set (0.01 sec)

mysql>
```

7) **Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.**

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE TEACHER (
    ->     TID VARCHAR(10),
    ->     Subject VARCHAR(30),
    ->     Hobby VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER VALUES
    -> ('T01','DBMS','Cricket'),
    -> ('T01','DBMS','Music'),
    -> ('T01','DSA','Cricket'),
    -> ('T01','DSA','Music'),
    -> ('T02','Networks','Reading'),
    -> ('T02','AI','Reading');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER;
+------+----------+---------+
| TID  | Subject  | Hobby   |
+------+----------+---------+
| T01  | DBMS     | Cricket |
| T01  | DBMS     | Music   |
| T01  | DSA      | Cricket |
| T01  | DSA      | Music   |
| T02  | Networks | Reading |
| T02  | AI       | Reading |
+------+----------+---------+
6 rows in set (0.00 sec)
```

MULTI-VALUED DEPENDENCIES

- TID →→ Subject
- TID →→ Hobby

DECOMPOSE INTO 4NF:

```
mysql> CREATE TABLE TEACHER_SUBJECT (
    ->     TID VARCHAR(10),
    ->     Subject VARCHAR(30),
    ->     PRIMARY KEY (TID, Subject)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO TEACHER_SUBJECT VALUES
    -> ('T01','DBMS'),
    -> ('T01','DSA'),
    -> ('T02','Networks'),
    -> ('T02','AI');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+----------+
| TID | Subject  |
+-----+----------+
| T01 | DBMS     |
| T01 | DSA      |
| T02 | AI       |
| T02 | Networks |
+-----+----------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE TEACHER_HOBBY (
    ->     TID VARCHAR(10),
    ->     Hobby VARCHAR(30),
    ->     PRIMARY KEY (TID, Hobby)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER_HOBBY VALUES
    -> ('T01','Cricket'),
    -> ('T01','Music'),
    -> ('T02','Reading');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+---------+
| TID | Hobby   |
+-----+---------+
| T01 | Cricket |
| T01 | Music   |
| T02 | Reading |
+-----+---------+
3 rows in set (0.00 sec)
```

- TID →→ Subject
- TID →→ Hobby

Therefore, the original relation contains independent multi-valued dependencies.
Decomposition:

- TEACHER_SUBJECT(TID, Subject)
- TEACHER_HOBBY(TID, Hobby)

8) Demonstrate how join dependencies lead to redundancy.Decompose a given relation to 5NF (Project-Join Normal Form).

```
Database changed
mysql> CREATE TABLE SUPPLIER_PRODUCT_PORT (
    ->     Supplier VARCHAR(10),
    ->     Product VARCHAR(10),
    ->     Port VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SUPPLIER_PRODUCT_PORT VALUES
    -> ('S01','P01','Chennai'),
    -> ('S01','P01','Mumbai'),
    -> ('S01','P02','Chennai'),
    -> ('S01','P02','Mumbai'),
    -> ('S02','P01','Chennai'),
    -> ('S02','P01','Mumbai');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SUPPLIER_PRODUCT_PORT;
+----------+---------+---------+
| Supplier | Product | Port    |
+----------+---------+---------+
| S01      | P01     | Chennai |
| S01      | P01     | Mumbai  |
| S01      | P02     | Chennai |
| S01      | P02     | Mumbai  |
| S02      | P01     | Chennai |
| S02      | P01     | Mumbai  |
+----------+---------+---------+
6 rows in set (0.00 sec)
```

DECOMPOSITION INTO 5NF

```
mysql> CREATE TABLE SUPPLIER_PRODUCT (
    ->     Supplier VARCHAR(10),
    ->     Product VARCHAR(10),
    ->     PRIMARY KEY (Supplier, Product)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SUPPLIER_PRODUCT VALUES
    -> ('S01','P01'),
    -> ('S01','P02'),
    -> ('S02','P01');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SUPPLIER_PRODUCT;
+----------+---------+
| Supplier | Product |
+----------+---------+
| S01      | P01     |
| S01      | P02     |
| S02      | P01     |
+----------+---------+
3 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE SUPPLIER_PORT (
    ->     Supplier VARCHAR(10),
    ->     Port VARCHAR(20),
    ->     PRIMARY KEY (Supplier, Port)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SUPPLIER_PORT VALUES
    -> ('S01','Chennai'),
    -> ('S01','Mumbai'),
    -> ('S02','Chennai'),
    -> ('S02','Mumbai');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SUPPLIER_PORT;
+----------+---------+
| Supplier | Port    |
+----------+---------+
| S01      | Chennai |
| S01      | Mumbai  |
| S02      | Chennai |
| S02      | Mumbai  |
+----------+---------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE PRODUCT_PORT (
    ->     Product VARCHAR(10),
    ->     Port VARCHAR(20),
    ->     PRIMARY KEY (Product, Port)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO PRODUCT_PORT VALUES
    -> ('P01','Chennai'),
    -> ('P01','Mumbai'),
    -> ('P02','Chennai'),
    -> ('P02','Mumbai');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM PRODUCT_PORT;
+---------+---------+
| Product | Port    |
+---------+---------+
| P01     | Chennai |
| P01     | Mumbai  |
| P02     | Chennai |
| P02     | Mumbai  |
+---------+---------+
4 rows in set (0.00 sec)
```

Original Relation:

- SUPPLIER_PRODUCT_PORT(Supplier, Product, Port)

Decomposition:

- SUPPLIER_PRODUCT(Supplier, Product)
- SUPPLIER_PORT(Supplier, Port)
- PRODUCT_PORT(Product, Port)

The three relations remove the redundancy caused by the independent associations and represent the join dependency separately.

9) **Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.**

```
mysql> CREATE TABLE COLLEGE_COURSE (
    ->     StudentID VARCHAR(10),
    ->     StudentName VARCHAR(30),
    ->     CourseID VARCHAR(10),
    ->     CourseName VARCHAR(30),
    ->     FacultyID VARCHAR(10),
    ->     FacultyName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COLLEGE_COURSE VALUES
    -> ('S101','Arun','C01','DBMS','F01','Dr.Ravi'),
    -> ('S102','Meena','C01','DBMS','F01','Dr.Ravi'),
    -> ('S103','Kiran','C02','Networks','F02','Dr.Suresh'),
    -> ('S104','Ravi','C03','AI','F03','Dr.Meena');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE_COURSE;
+-----------+-------------+----------+------------+-----------+-------------+
| StudentID | StudentName | CourseID | CourseName | FacultyID | FacultyName |
+-----------+-------------+----------+------------+-----------+-------------+
| S101      | Arun        | C01      | DBMS       | F01       | Dr.Ravi     |
| S102      | Meena       | C01      | DBMS       | F01       | Dr.Ravi     |
| S103      | Kiran       | C02      | Networks   | F02       | Dr.Suresh   |
| S104      | Ravi        | C03      | AI         | F03       | Dr.Meena    |
+-----------+-------------+----------+------------+-----------+-------------+
4 rows in set (0.00 sec)

mysql>
```

Functional Dependencies

- StudentID → StudentName
- CourseID → CourseName, FacultyID
- FacultyID → FacultyName

Therefore, there is a transitive dependency:
CourseID → FacultyID → FacultyName
be lost.

DECOMPOSE INTO 3NF:

```
mysql> CREATE TABLE COLLEGE_STUDENT (
    ->     StudentID VARCHAR(10) PRIMARY KEY,
    ->     StudentName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COLLEGE_STUDENT VALUES
    -> ('S101','Arun'),
    -> ('S102','Meena'),
    -> ('S103','Kiran'),
    -> ('S104','Ravi');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE_STUDENT;
+-----------+-------------+
| StudentID | StudentName |
+-----------+-------------+
| S101      | Arun        |
| S102      | Meena       |
| S103      | Kiran       |
| S104      | Ravi        |
+-----------+-------------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE COLLEGE_FACULTY (
    ->     FacultyID VARCHAR(10) PRIMARY KEY,
    ->     FacultyName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COLLEGE_FACULTY VALUES
    -> ('F01','Dr.Ravi'),
    -> ('F02','Dr.Suresh'),
    -> ('F03','Dr.Meena');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE_FACULTY;
+-----------+-------------+
| FacultyID | FacultyName |
+-----------+-------------+
| F01       | Dr.Ravi     |
| F02       | Dr.Suresh   |
| F03       | Dr.Meena    |
+-----------+-------------+
3 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE COLLEGE_COURSE_3NF (
    ->     CourseID VARCHAR(10) PRIMARY KEY,
    ->     CourseName VARCHAR(30),
    ->     FacultyID VARCHAR(10),
    ->     FOREIGN KEY (FacultyID)
    ->     REFERENCES COLLEGE_FACULTY(FacultyID)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO COLLEGE_COURSE_3NF VALUES
    -> ('C01','DBMS','F01'),
    -> ('C02','Networks','F02'),
    -> ('C03','AI','F03');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE_COURSE_3NF;
+----------+------------+-----------+
| CourseID | CourseName | FacultyID |
+----------+------------+-----------+
| C01      | DBMS       | F01       |
| C02      | Networks   | F02       |
| C03      | AI         | F03       |
+----------+------------+-----------+
3 rows in set (0.00 sec)
```

Original:

COLLEGE_COURSE

- (StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

3NF Decomposition:

- COLLEGE_STUDENT(StudentID, StudentName)
- COLLEGE_COURSE_3NF(CourseID, CourseName, FacultyID)
- COLLEGE_FACULTY(FacultyID, FacultyName)

10) Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

```
mysql> CREATE TABLE RESEARCH (
    ->     ResearcherID VARCHAR(10),
    ->     ProjectID VARCHAR(10),
    ->     ProjectName VARCHAR(30),
    ->     LabID VARCHAR(10),
    ->     LabName VARCHAR(30),
    ->     PRIMARY KEY (ResearcherID)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RESEARCH VALUES
    -> ('R101','P101','AI Research','L01','AI Lab'),
    -> ('R102','P102','Robotics','L02','Robotics Lab'),
    -> ('R103','P103','Cyber Security','L03','Security Lab'),
    -> ('R104','P104','Cloud Computing','L04','Cloud Lab');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCH;
+--------------+-----------+-----------------+-------+--------------+
| ResearcherID | ProjectID | ProjectName     | LabID | LabName      |
+--------------+-----------+-----------------+-------+--------------+
| R101         | P101      | AI Research     | L01   | AI Lab       |
| R102         | P102      | Robotics        | L02   | Robotics Lab |
| R103         | P103      | Cyber Security  | L03   | Security Lab |
| R104         | P104      | Cloud Computing | L04   | Cloud Lab    |
+--------------+-----------+-----------------+-------+--------------+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- ResearcherID → ProjectID
- ProjectID → ProjectName, LabID
- LabID → LabName

- Candidate Key = ResearcherID

SQL VERIFICATION

```
mysql> SELECT ResearcherID,
    ->        COUNT(DISTINCT ProjectID) AS Project_Count,
    ->        COUNT(DISTINCT ProjectName) AS ProjectName_Count,
    ->        COUNT(DISTINCT LabID) AS Lab_Count,
    ->        COUNT(DISTINCT LabName) AS LabName_Count
    -> FROM RESEARCH
    -> GROUP BY ResearcherID;
+--------------+---------------+-------------------+-----------+---------------+
| ResearcherID | Project_Count | ProjectName_Count | Lab_Count | LabName_Count |
+--------------+---------------+-------------------+-----------+---------------+
| R101         |             1 |                 1 |         1 |             1 |
| R102         |             1 |                 1 |         1 |             1 |
| R103         |             1 |                 1 |         1 |             1 |
| R104         |             1 |                 1 |         1 |             1 |
+--------------+---------------+-------------------+-----------+---------------+
4 rows in set (0.00 sec)
```